

Week 0 — Get Ready to Work

GitHub, coding agents, and Colab/Python

Shane Conway

2026-08-13

Overview

Week 0 is a setup hour. By the end, you will be able to:

- ▶ Introduce some basic tools (e.g. Markdown, coding editor)
- ▶ Get your own copy of the course repository and pull, commit, and push with Git and GitHub
- ▶ (Optional) Run a coding agent (Claude Code or Codex) and inspect what it proposes before accepting it
- ▶ Run the setup notebook in Colab or local Python from a clean kernel

None of it is graded. All of it is assumed from Week 1 onward.

All of these materials are available on the course github site.

When in doubt, use AI.

This is a course about ML and AI, and we encourage you to use AI liberally (but check it's work!).

Basic tools

Install the tools you will use

Install from the official pages (versions change):

- ▶ **Git**: <https://git-scm.com> (a GitHub account does not install it)
- ▶ **Python 3.10+**: <https://python.org> (or use Colab and skip this)
- ▶ **Node.js LTS**: <https://nodejs.org> (only for the optional coding agents)
- ▶ **Jupyter**: <https://jupyter.org/> all work will be run and submitted in jupyter notebooks.
`brew install jupyterlab`

On Windows, the commands are often `py` and `python`, not `python3`.

- ▶ **Anaconda / Miniconda** is an alternative Python distribution; either the `python.org` installer or a `conda` install is fine.
- ▶ **A package manager saves the clicking.** On macOS, Homebrew installs everything from one line — `brew install git python node` — and on Windows, winget does the same.

Terminal

Get comfortable at a Unix-style command line (macOS Terminal, Linux, or WSL/PowerShell on Windows). A handful of commands cover most day-to-day work:

Commands	What they do
<code>pwd ls cd</code>	where am I, what is here, move around
<code>mkdir touch</code>	make a directory, create a file
<code>cat vi</code>	print a file, edit one in place
<code>grep sed awk</code>	search and transform text
<code>chmod top</code>	file permissions, running processes

A CLI is a command-line tool; most respond to `--help` (or `man <name>`) when you forget the arguments.

An editor for code and notes

You will read and write code, notebooks, Markdown, and small config files, so a capable editor is worth setting up early.

- ▶ **VS Code** — free, cross-platform; strong Python, Jupyter, and Git support, and it runs Claude Code or Codex in its built-in terminal.
- ▶ **PyCharm** - common if just working in python
- ▶ **TextMate 2** (macOS), or any editor you already prefer.
- ▶ **Vi or Emacs**

Open the *cloned folder*, not one file at a time. The editor then shows the whole project — files, diffs, and search — in one place.

Markdown: plain text with light structure

READMEs, notebook cells, homework write-ups, and agent instructions are all Markdown: plain text with a few marks.

```
# Heading      **bold**    *italic*    `code`  
- a bullet    [a link](https://...)
```

The same text renders on GitHub, in notebooks, and on these slides. Write prose that reads plainly; add structure only when it clarifies.

Quarto: one source, many outputs

These slides and the course book are built with Quarto: a plain-text .qmd (Markdown plus code) renders to HTML, PDF, slides, or a website from one source.

```
quarto render report.qmd      # -> report.html or report.pdf  
quarto preview report.qmd    # live preview while you edit
```

- ▶ Prose, code, and results live in one document; re-running reproduces the outputs.
- ▶ Optional for the course, useful for a reproducible final-project write-up.
- ▶ Install from <https://quarto.org>; VS Code has a Quarto extension.

LaTeX

As a researcher, it is worth being aware of LaTeX, the typesetting language for mathematics. You do not write papers in it for this course, but the *same* math syntax works in Markdown, notebooks, and these slides. Wrap an inline symbol in single dollar signs and a displayed equation in double:

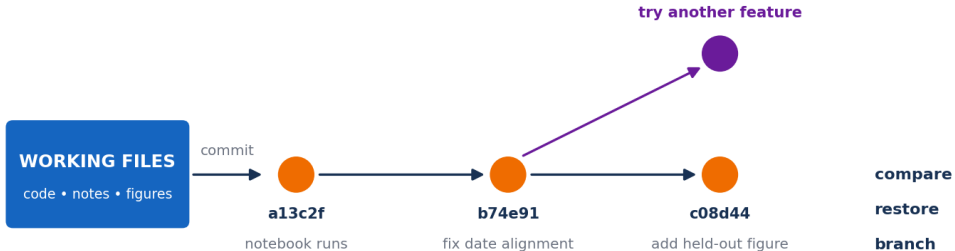
Inline: the simple return is $r_t = p_t / p_{t-1} - 1$.

$$\hat{y} = \beta_0 + \sum_{j=1}^p \beta_j x_j$$

which renders the symbol in the sentence and the fitted line beneath it.

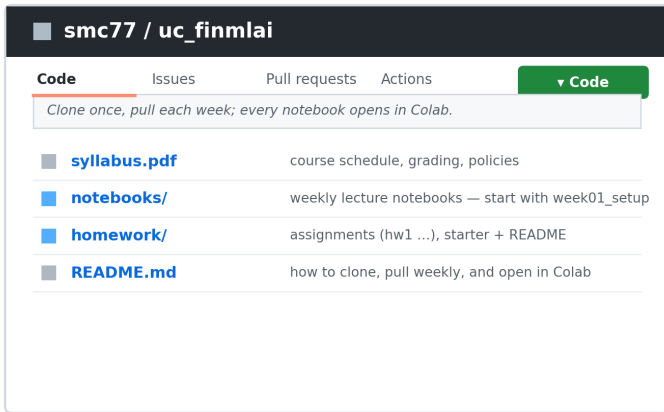
GitHub and the course repository

Git records a history of your work



Git is version-control software. It records deliberate snapshots called *commits*, then lets you compare, restore, or branch from them.

Get access to the course repository



■ **smc77 / uc_finmlai**

Code Issues Pull requests Actions ▼ Code

Clone once, pull each week; every notebook opens in Colab.

■ syllabus.pdf	course schedule, grading, policies
■ notebooks/	weekly lecture notebooks — start with week01_setup
■ homework/	assignments (hw1 ...), starter + README
■ README.md	how to clone, pull weekly, and open in Colab

Sign in to GitHub and open the repository above. Read the `README.md`, then find the setup notebook, the syllabus, and Homework 1.

Start from the template

```
# GitHub: open uc_finmlai, click "Use this template"
#   -> Create a new repository, Visibility: Private
git clone https://github.com/<you>/<your-repo>.git
cd <your-repo>
git status
git remote -v          # origin is YOUR repo -- you can push here
```

“Use this template” gives you your own private repository. HTTPS is the simplest first path; SSH and the GitHub CLI are useful once authentication is configured.

Template, clone, or download?

Approach	Your own repo	Push your work	Pull updates
use this template	yes (private)	yes	via upstream
clone course repo	no	no	yes
download ZIP	no	no	no

Use the template so `origin` is yours (next slide). A ZIP is a copy of files, not a working Git repository.

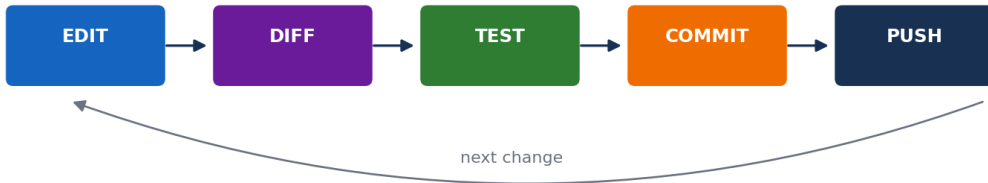
Sync with the course; push your own work

```
git remote add upstream https://github.com/smc77/uc_finmlai.git
git pull upstream main          # each week, collect new material
# ... do your work in homework/ and work/, then:
git add -A && git commit -m "week 1"
git push                        # to origin -- your private repo
```

origin is your private repo; upstream is the course source. Keep your work in homework/ and work/ so weekly pulls merge cleanly. Flip the repo to public at the end as your portfolio. Assignments are submitted through Canvas.

The daily Git loop

A commit is the address of a tested result



The five commands

```
git pull --rebase
git status
git diff
git add path/to/file
git commit -m "Explain why the result changed"
```

Then `git push` when the remote is yours. Read `status` and `diff` before staging anything.

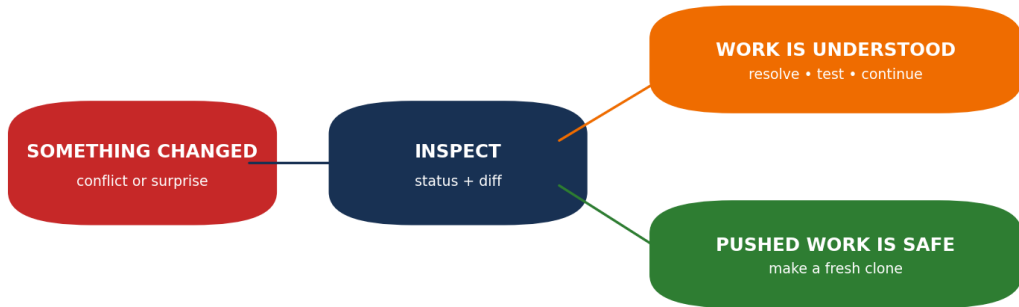
Recover an earlier version

```
git log --oneline          # list commits, newest first
git show <hash>:path/to/file # view a file as it was
git checkout <hash> -- path  # restore that file into your working tree
```

This is why a commit message should say *why* a result changed: the log is how you find the version that produced it.

When Git goes wrong

Git records evidence; it does not require improvisation under pressure



Do not force an unexplained state.

Keep local build files and secrets out

```
.venv/  
.env  
.ipynb_checkpoints/  
__pycache__/
```

Do not commit passwords, API keys, private data, or large generated files. Deleting a secret in a later commit does not remove it from history.

Authenticate once to push

Cloning a public repository needs nothing. Pushing needs a credential:

```
gh auth login      # GitHub CLI: a browser sign-in that configures HTTPS push
```

SSH keys are the alternative. Until this is set up, your commits live only on your machine, which is enough for the exercise below.

Break: prove the repository path

1. On GitHub, create your repo from the template (or confirm your clone).
2. Clone your repo; run `git status` and `git remote -v`.
3. Add the course repo as upstream.
4. Make one harmless note in a scratch file.
5. Inspect the diff and commit it. Required artifact: the commit hash.
6. Push the commit to origin (your own repo).

Keep: the commit hash and your repo's remote URL.

AI agents

What a coding agent can do

The agent proposes; the researcher remains responsible



The choice is optional

Claude Code

Anthropic terminal agent

`claude` in the repository

Anthropic account or supported enterprise
route

Codex

OpenAI terminal/editor/app agent

`codex` in the repository

ChatGPT login or API route

Neither is required for the assessed path. Accounts, access, and billing differ; use the option available to you.

Install Claude Code

Current documented npm path:

```
npm install -g @anthropic-ai/claude-code  
claude doctor  
cd <your-repo>  
claude
```

Do not use `sudo npm install -g`. Follow Anthropic's current setup page if the installer or authentication options have changed.

Install and sign in to Codex

Current npm path:

```
npm install -g @openai/codex  
codex login  
cd <your-repo>  
codex
```

`codex login` opens the browser sign-in flow. API-key use is a separate, usage-billed route and is not needed merely because you have a ChatGPT login.

Start inside the repository

```
pwd  
git status  
claude          # or: codex
```

The working directory defines what the agent can see. Begin with a clean or understood working tree so its changes are distinguishable from yours.

Tell the agent the house rules

An agent works better when the repository states its conventions once, in a file it reads on its own instead of a prompt you retype.

- ▶ `CLAUDE.md` (Claude Code) or `AGENTS.md` (Codex): instructions at the repo root — how to run things, the data-and-time rules, what not to touch.
- ▶ A **skill** packages a repeatable task — a checklist the agent invokes by name.

Write the standing rule in the repo, not in every prompt. A one-off decision, like the forecast timestamp for one experiment, still belongs in that task.

Give it one bounded task

```
Read README.md and lectures/week00/setup.ipynb.  
Explain how the notebook finds finmlsim.  
Do not edit files.  
Name the command that would verify your answer.
```

Scope, relevant files, permitted actions, and verification belong in the request.

Inspect before accepting

```
git status  
git diff
```

Ask:

- ▶ Did it change only the intended files?
- ▶ Can you explain every changed line?
- ▶ Did it preserve the time and data assumptions?
- ▶ Did the verification actually run?

The agent is not the evidence

An agent can produce code that runs while:

- ▶ fitting a transform before the time split;
- ▶ using revised or future data;
- ▶ changing the sample along with the model;
- ▶ citing a source that does not support the claim.

You remain responsible for the analysis, interpretation, citations, and disclosure required by the syllabus. Agents are encouraged on homework with disclosure; they are not permitted on quizzes.

Gemini Notebook (NotebookLM)

NotebookLM (<https://notebooklm.google.com>) is a research assistant grounded in *sources you upload* — PDFs, slides, notes, a paper — rather than the open web.

- ▶ It answers questions, summarizes, and builds study aids (including audio overviews) that cite back to your own documents, so its claims are easier to check than a general chatbot's.
- ▶ Useful for interrogating the assigned readings and your own notes.
- ▶ Same discipline as any agent: a way to read faster, not a substitute for reading, and it does not relieve you of citing the original source.

Break: run the review loop

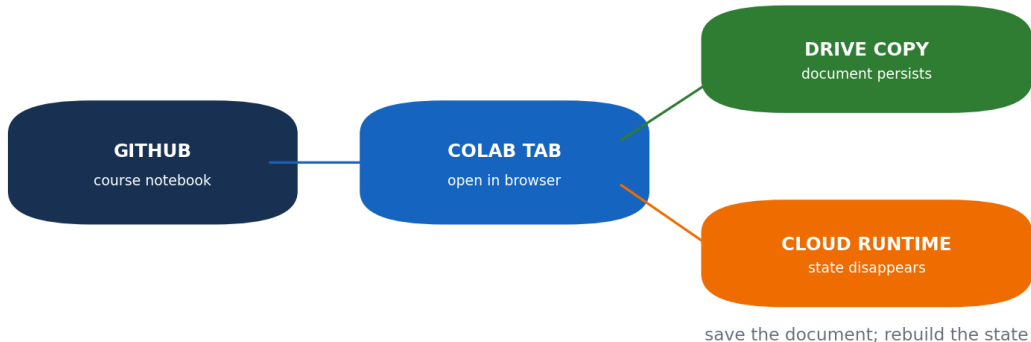
1. Start Claude Code or Codex in a practice repository.
2. Give it the bounded, read-only task from the previous slide.
3. Ask for one harmless documentation improvement.
4. Inspect `git diff`.
5. Accept, revise, or reject it; run the relevant check.

Keep: the prompt, the diff, and one sentence explaining your decision.

Colab and Python

Colab is the supported zero-install path

The notebook and the machine running it are different objects



Open the setup notebook from GitHub, then use `File > Save a copy in Drive`.

What Colab does not preserve

preserved	temporary runtime
notebook in Drive or GitHub	installed packages
saved notebook code/output	uploaded local files
repository history	variables and execution state

Every notebook therefore opens with a visible install cell, for example `!pip install finmlsim==0.4.1`, followed by its data-loading cells. A runtime reset is normal, not evidence that the notebook was saved incorrectly.

The local Python alternative

```
python3 --version          # Python 3.10+
python3 -m venv .venv
source .venv/bin/activate  # macOS/Linux
# .venv\Scripts\Activate.ps1 # Windows PowerShell

python -m pip install -r requirements.txt
python -m pip install -e .
```

One project-local environment keeps the course packages together.

finmlsim and where data comes from

`finmlsim` provides the simulators and metrics behind the course's figures.

```
pip install finmlsim==0.4.1      # pinned; also listed in requirements.txt
```

On Colab, reinstall it after each runtime reset. Course data (Fama-French, S&P 500, Treasury) is fetched by the setup notebook on first run, not committed to the repository, so an initial network call is expected.

Environment and kernel must agree

```
python -c "import sys; print(sys.executable)"
```

```
import sys  
print(sys.executable)
```

The terminal and notebook should identify the same environment. “I installed it” does not help a notebook running a different Python.

Verify imports directly

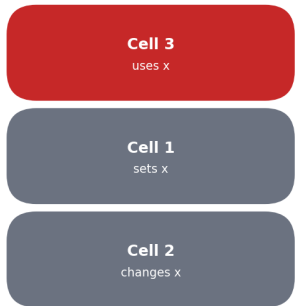
```
python -c "import numpy, pandas, sklearn, finmlsim; \  
print(numpy.__version__, pandas.__version__, sklearn.__version__)"
```

Read failures from the final traceback line. Preserve the exception, Python path, and command before trying a repair. When stuck, post those three, plus the output of `python -c "import sys; print(sys.executable)"`, to the Canvas thread.

A notebook has hidden state

A notebook is reproducible only when execution order is part of the evidence

Hidden state



works only in one remembered session

Restart & Run All



reconstructs the result from a blank kernel

The clean-run test

Can the notebook rebuild its own result?



A saved output is evidence only when the path to it survives a restart.

The Python baseline

You should be able to read and modify:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
returns = pd.Series(rng.normal(0.0003, 0.01, 1000))
rolling_vol = returns.rolling(20).std()
sharpe = np.sqrt(252) * returns.mean() / returns.std()
```

The refresher lives in the setup notebook and linked readings.

pandas: Series, DataFrame, and the index

```
import pandas as pd

px = pd.read_csv("prices.csv", index_col="date", parse_dates=True)
px = px.sort_index()           # a time index must be sorted
px["AAPL"]                    # one column: a Series
px.loc["2020-03":"2020-06"]   # select by label (a date range)
px.iloc[0]                    # select by position (the first row)
```

The index is the clock. `.loc` selects by label; `.iloc` by position. Confusing the two is a common silent error.

pandas without look-ahead

```
r = px["AAPL"].pct_change()
feature = r.rolling(20, min_periods=20).mean().shift(1)
target  = r.shift(-1)
panel   = feature.to_frame("mom").join(target.rename("fwd")).dropna()
```

.shift(1) lags the feature so it uses only the past; .shift(-1) makes next period's return the target; join aligns on the date index, so rows from different dates never line up by accident.

Time-indexed data needs a clock

```
prices.index.is_monotonic_increasing
prices.index.is_unique

returns = prices.pct_change()
feature_t = returns.rolling(20).mean()
target_t = returns.shift(-1)
```

State when information is observed, when a decision is made, and which later return it earns.

Readiness check

You are ready for Week 1 when you can:

- ▶ pull the repository and identify its remote;
- ▶ run the setup notebook from a blank kernel;
- ▶ print the Python executable and package versions;
- ▶ inspect array shape and a DataFrame time index;
- ▶ read a traceback and test a repair;
- ▶ inspect an agent's diff, if you choose to use one.

The agent is optional. The reproducible path is not.

Arrive at Week 1 with one clean notebook run and a working Git pull.